DIGT2107: Practice of Software Development

JWTOffline

Iteration 2 (Deliverable 1.2): Detailed User Stories, Requirements, and Initial Prototype

---

Course: DIGT2107, Fall Term 2026

Instructor: Dr. May Haidar

Team Number: [Team #]

Team Members: Isa Din, Steven Passynkov, Osmond Jiang

Submitted: October 11, 2026

Repository: https://github.com/isad07/JWToffline (tag `ITR1.2`)

Contents

1. Introduction

2. Functional and Non-Functional Requirements

- 2.1 Functional Requirements
- 2.2 Non-Functional Requirements
- 2.3 Traceability Matrix

3. Use Cases

- 3.1 Use Case Diagram
- 3.2 Use Case Descriptions

4. Detailed User Stories

- 4.1 to 4.5 Stories by feature category

5. Initial Prototype Overview

- 5.1 Running the Prototype
- 5.2 Screens
- 5.3 Testing So Far
- 5.4 Developer Tasks

6. Documentation Updates

- 6.1 System Architecture
- 6.2 Planning Document and Log

7. Iteration Plan and Backlog

- 7.1 Iteration Plan
- 7.2 Updated Backlog

1. Introduction

Project Name: JWTOffline

Team Number: [Team #]

Team Members: Isa Din, Steven Passynkov, Osmond Jiang

In Iteration 1 we described JWTOffline as a desktop app that lets developers decode, build, and verify JSON Web Tokens without pasting them into a website. This iteration turns that vision into something concrete. We wrote down exactly what the system has to do, how it should behave under real use, who interacts with it, and how each feature breaks into stories small enough to build in a sprint.

We also started building. The prototype tagged `ITR1.2` already decodes tokens, signs new ones with six algorithms, checks signatures, and keeps a workspace of saved tokens. The workspace uses mock in memory data for now, which the spec allows. 47 unit tests cover the logic underneath the screens.

How this document is organized

- **Section 2** lists functional and non-functional requirements, then a traceability matrix that links every requirement to its user story, use case, and test class.
- **Section 3** has the UML use case diagram and a full description of each use case.
- **Section 4** breaks the four feature categories from Iteration 1 into detailed user stories with acceptance criteria, priority, story points, and hour estimates.
- **Section 5** walks through the prototype: what works, how to run it, the screens, and the developer task breakdown.
- **Section 6** covers the architecture and records how our plan changed since Iteration 1.
- **Section 7** is the iteration plan and the prioritized backlog for what comes next.

2. Functional and Non-Functional Requirements

Requirements are written as "the system shall" statements so each one can be tested. Each functional requirement lists the user story it came from. Status of each one in the prototype is shown in the traceability matrix in 2.3.

2.1 Functional Requirements

ID	Requirement	Priority	Story
FR-1	The system shall accept a token pasted into the decoder and ignore surrounding whitespace and a leading "Bearer " prefix.	High	US-1.1
FR-2	The system shall split a token into header, payload, and signature and Base64URL decode the header and payload locally.	High	US-1.1
FR-3	The system shall display the decoded header and payload as indented, readable JSON, keeping the original key order.	High	US-1.1
FR-4	The system shall show a plain language error, and never crash, when a token has the wrong number of parts, bad Base64URL, or invalid JSON.	High	US-1.4
FR-5	The system shall convert the exp, iat, and nbf claims into readable local date and time.	High	US-1.3
FR-6	The system shall show a status banner saying whether the token is valid, expired, not yet valid, or has no expiry, and how long until or since that point.	High	US-1.3
FR-7	The system shall show the signing algorithm named in the header and the signature length.	Medium	US-1.2
FR-8	The system shall let the user copy the header, payload, or generated token to the clipboard with one click.	Medium	US-1.2, US-2.3
FR-9	The system shall let the user type header and payload JSON and shall validate it before signing, pointing to the position of any error.	High	US-2.1
FR-10	The system shall support HS256, HS384, HS512, RS256, RS384, and RS512 and shall set the alg and typ header values automatically.	High	US-2.2
FR-11	The system shall sign a new token with an HMAC secret or a PKCS#8 PEM RSA private key and output it in compact form.	High	US-2.2
FR-12	The system shall let the user open a newly generated token directly in the decoder.	Medium	US-2.3
FR-13	The system shall verify HMAC signed tokens against a secret entered by the user.	High	US-3.1
FR-14	The system shall verify RSA signed tokens against a PEM public key entered by the user.	Medium	US-3.2
FR-15	The system shall clearly report a pass or fail result with the reason, and shall refuse tokens using alg "none" or an unsupported algorithm.	High	US-3.3
FR-16	The system shall let the user send the token currently in the decoder straight to the verify screen.	Medium	US-3.1
FR-17	The system shall let the user save a decoded token to the workspace with a label.	Medium	US-4.1

ID	Requirement	Priority	Story
FR-18	The system shall list saved tokens with their algorithm, status, and save time, and filter them by label.	Medium	US-4.2
FR-19	The system shall let the user rename or delete a saved token.	Low	US-4.3
FR-20	The system shall open a saved token in the decoder with a double click or button.	Medium	US-4.2
FR-21	The system shall keep the 20 most recent tokens decoded, created, or verified in the session, newest first and without duplicates, and show them on the Home screen.	Low	US-4.4
FR-22	The system shall store the workspace in a local file so saved tokens survive closing the app.	Medium	US-4.5
FR-23	The system shall provide a single window with side navigation between Home, Decode, Encode, Verify, and Workspace, keeping each screen's input when the user switches away.	High	US-5.1

2.2 Non-Functional Requirements

These describe how well the system has to work, not what it does. Each one has a target we can measure and a way we plan to check it.

ID	Quality	Requirement and target	How we will check it
NFR-1	Privacy	The app makes zero network calls. All decoding, signing, and verification happens on the user's machine. Secrets and keys are never written to disk.	Code review: no networking classes used. Run with network disabled.
NFR-2	Performance	Decoding or verifying a token up to 8 KB completes in under 100 ms. The decoder updates as the user types with no visible lag. The app starts in under 3 seconds on a typical student laptop.	Timed unit test (Iteration 3), manual stopwatch check.
NFR-3	Usability	A developer who has never seen the app can decode their first token within 30 seconds without instructions. Errors are written in plain language and say what to fix.	Hallway test with 3 classmates in Iteration 3.
NFR-4	Security	Signature comparison uses a constant time check. Tokens with alg "none" are never reported as verified. The selected algorithm, not the token header, decides how a token is signed.	SignatureVerifierTest, JwtEncoderTest.
NFR-5	Reliability	No input typed or pasted by the user can crash the app. Every failure ends in a message on screen.	Negative unit tests for every parser.
NFR-6	Portability	Runs on Windows, macOS, and Linux with JDK 17 or newer. No native code and no runtime libraries beyond the JDK and JavaFX.	CI build on Ubuntu, manual run on Windows and macOS.
NFR-7	Maintainability and Scalability	Logic lives in a core package with no UI code so new algorithms (for example ES256) or new screens can be added without touching existing ones. Storage sits behind an interface so the in memory store can be swapped for a file store. Core and workspace code keep at least 80% line coverage.	JaCoCo report in CI.

ID	Quality	Requirement and target	How we will check it
NFR-8	Accessibility	All actions are reachable by keyboard, text meets WCAG AA contrast, and status is shown with words as well as color.	Manual check in Iteration 3.

2.3 Traceability Matrix

This matrix lets anyone follow a requirement back to the story that asked for it and forward to the test that proves it works. Test class names match the files in `src/test/java`.

Requirement	User story	Use case	Verified by	Prototype status
FR-1, FR-2, FR-3	US-1.1	UC1	JwtDecoderTest, JsonTest	Done
FR-4	US-1.4	UC1	JwtDecoderTest, JsonTest	Done
FR-5, FR-6	US-1.3	UC2	ClaimInspectorTest	Done
FR-7, FR-8	US-1.2	UC1	JwtDecoderTest (UI copy is manual)	Done
FR-9	US-2.1	UC3	JwtEncoderTest, JsonTest	Done
FR-10, FR-11	US-2.2	UC3, UC4	JwtEncoderTest	Done
FR-12	US-2.3	UC3	Manual UI check	Done
FR-13, FR-16	US-3.1	UC5	SignatureVerifierTest	Done
FR-14	US-3.2	UC5	SignatureVerifierTest	Done
FR-15	US-3.3	UC5	SignatureVerifierTest	Done
FR-17	US-4.1	UC6	InMemoryTokenRepositoryTest	Done (mock data)
FR-18, FR-20	US-4.2	UC7	InMemoryTokenRepositoryTest	Done (mock data)
FR-19	US-4.3	UC6	InMemoryTokenRepositoryTest	Done (mock data)
FR-21	US-4.4	UC8	TokenHistoryTest	Done
FR-22	US-4.5	UC6	Planned: FileTokenRepositoryTest	Iteration 3
FR-23	US-5.1	All	Manual UI check	Done
NFR-4, NFR-5	US-1.4, US-3.3	UC1, UC5	SignatureVerifierTest, JwtDecoderTest	Done
NFR-7	All	n/a	JaCoCo report in CI	Set up

3. Use Cases

3.1 Use Case Diagram

There are three human actors. The **Developer** is our main user and touches every feature. The **QA Engineer** mostly reads and checks tokens while testing login flows, so they use decoding, verification, and the workspace. The **Student / Learner** uses the decoder and encoder to see how tokens are put together. **Local Storage** is a secondary system actor that holds the workspace; in the prototype it is an in memory stub.

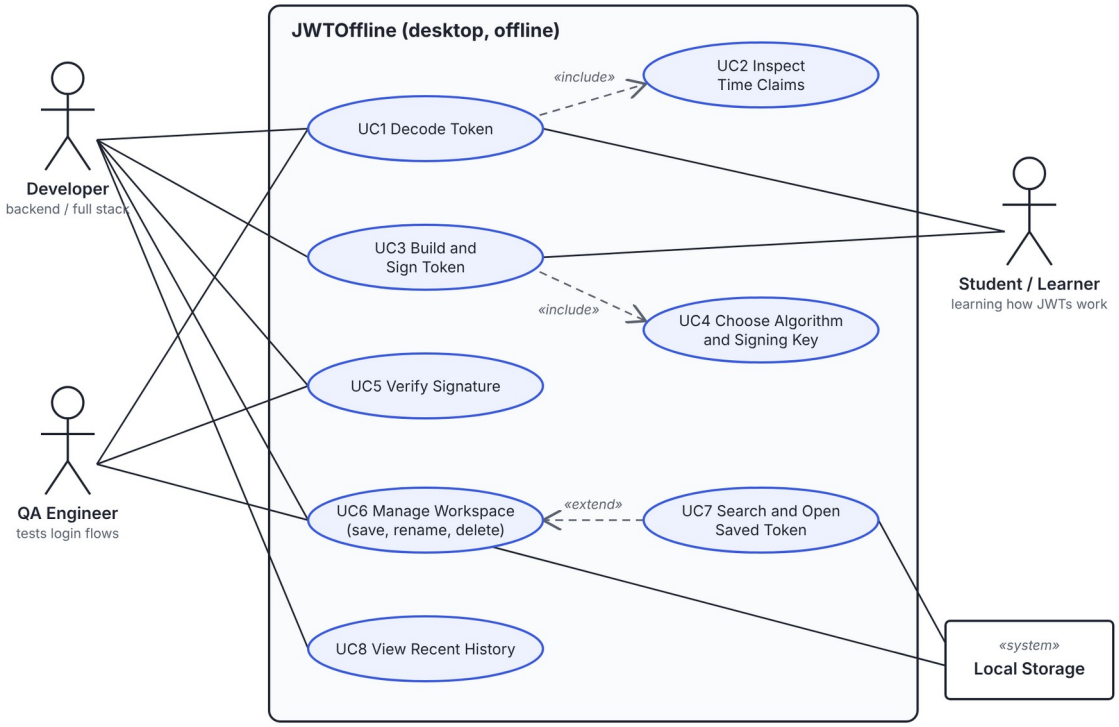


Figure 1. UML use case diagram for JWTOffline

3.2 Use Case Descriptions

UC1: Decode Token

Actors	Developer, QA Engineer, Student / Learner
Description	The user pastes a JWT and sees what is inside it without sending it anywhere.
Preconditions	The app is open. The user has a token, for example copied from browser dev tools or an API response.

Main flow	1. The user selects "Decode" from the side menu. 2. The user pastes the token into the input box. 3. The system trims whitespace and any "Bearer " prefix and splits the token on its dots. 4. The system Base64URL decodes the header and payload and shows them as formatted JSON. 5. The system shows the algorithm and signature length, then runs UC2 to show time claims and the status banner. 6. The system adds the token to recent history.
Alternative flows	1. 3a. The token does not have three parts, or a part is not valid Base64URL or JSON: the system shows a red message saying which part failed and clears the output panes. The user fixes the input and the flow restarts at step 3. 2. 6a. The user clicks "Save to workspace": UC6 runs. 3. 6b. The user clicks "Verify signature": the token is passed to UC5.
Postconditions	The decoded header and payload are on screen. Nothing has left the computer.

UC2: Inspect Time Claims (included by UC1)

Actors	Developer, QA Engineer (through UC1)
Description	The system explains when the token was issued, when it expires, and whether it is usable right now.
Preconditions	A token has been decoded successfully in UC1.
Main flow	1. The system reads exp, iat, and nbf from the payload if they are numbers. 2. The system converts each one to local date and time and lists it under "Time claims". 3. The system compares exp and nbf with the current time. 4. The system shows a green, red, or amber banner such as "Valid, expires in 2 hours" or "Expired 3 days ago".
Alternative flows	1. 1a. No time claims are present: the system says so and shows an amber "No expiry set" banner. 2. 3a. nbf is in the future: the banner reads "Not valid yet, starts in …".
Postconditions	The user knows whether the token is expired without doing any timestamp math.

UC3: Build and Sign Token

Actors	Developer, Student / Learner
Description	The user creates a new signed test token from claims they type in.
Preconditions	The app is open. For RSA, the user has a PKCS#8 PEM private key.

Main flow	1. The user selects "Encode" from the side menu. 2. The system shows a starter header and a payload with iat set to now and exp one hour later. 3. The user edits the header and payload JSON. 4. The user chooses an algorithm and key (UC4). 5. The user clicks "Generate token". 6. The system validates the JSON, sets alg and typ in the header, signs the token, and shows it with a green confirmation. 7. The user copies the token or clicks "Open in decoder".
Alternative flows	1. 6a. The JSON is invalid: the system shows "Invalid JSON" with the character position and does not produce a token. 2. 6b. The key is empty or the RSA key cannot be read: the system explains what kind of key it expected.
Postconditions	A valid compact JWT is displayed and added to recent history.

UC4: Choose Algorithm and Signing Key (included by UC3)

Actors	Developer, Student / Learner (through UC3)
Description	The user picks how the token will be signed.
Preconditions	The user is on the Encode screen.
Main flow	1. The user opens the algorithm drop down and picks one of HS256, HS384, HS512, RS256, RS384, or RS512. 2. For HS algorithms the user types a shared secret. For RS algorithms the user pastes a PEM private key. 3. The system keeps the choice for when the token is generated.
Alternative flows	2a. The user pastes a PKCS#1 key (BEGIN RSA PRIVATE KEY): on generate, the system explains that PKCS#8 is needed.
Postconditions	An algorithm and key are ready for signing.

UC5: Verify Signature

Actors	Developer, QA Engineer
Description	The user checks that a token was signed with the expected key and has not been edited.
Preconditions	The user has a token and the matching secret or RSA public key.
Main flow	1. The user selects "Verify", or clicks "Verify signature" on the Decode screen so the token is filled in. 2. The user pastes the secret or PEM public key. 3. The user clicks "Verify signature". 4. The system decodes the token, reads alg, and recomputes or checks the signature. 5. The system shows a green "Signature verified (HS256)" banner.

Alternative flows	1. 4a. alg is "none" or unsupported: the system refuses and says why. 2. 5a. The signature does not match: a red banner says the key is wrong or the token was changed. 3. 4b. The token is malformed: the decoder error is shown instead of a crash.
Postconditions	The user knows if the token can be trusted. A verified token is added to history.

UC6: Manage Workspace

Actors	Developer, QA Engineer, Local Storage
Description	The user keeps tokens they reuse, gives them labels, and removes old ones.
Preconditions	For saving, a token has been decoded in UC1. For rename or delete, at least one token is saved.
Main flow	1. From the Decode screen the user clicks "Save to workspace". 2. The system suggests the sub claim as the label and the user confirms or edits it. 3. The system stores the token with the label and save time in Local Storage. 4. On the Workspace screen the user selects a row and clicks "Rename" or "Delete". 5. The system updates Local Storage and refreshes the table.
Alternative flows	1. 2a. The user leaves the label blank: the token is saved as "Untitled token". 2. 4a. No row is selected: the system asks the user to select a token first. 3. 4b. The new label is blank: the rename is rejected with a message.
Postconditions	The workspace reflects the change. In the prototype it lasts until the app closes; in Iteration 3 it is saved to a file.

UC7: Search and Open Saved Token (extends UC6)

Actors	Developer, QA Engineer, Local Storage
Description	The user finds a saved token by name and reopens it.
Preconditions	The workspace contains at least one token.
Main flow	1. The user selects "Workspace" from the side menu. 2. The system lists saved tokens, newest first, with algorithm, status, and save time. 3. The user types part of a label in the search box and the list filters as they type. 4. The user double clicks a row or clicks "Open in decoder". 5. The system switches to Decode with the token loaded (UC1 from step 3).
Alternative flows	3a. Nothing matches: the table shows an empty list and the user clears the search.
Postconditions	The chosen token is decoded on screen.

UC8: View Recent History

Actors	Developer
Description	The user jumps back to a token they worked with earlier in the session.

Preconditions	At least one token was decoded, created, or verified this session.
Main flow	1. The user selects "Home". 2. The system shows the 20 most recent tokens, newest first. 3. The user double clicks one. 4. The system opens it in the decoder.
Alternative flows	2a. History is empty: the list shows a hint explaining what will appear there.
Postconditions	The selected token is decoded on screen.

4. Detailed User Stories

These stories break down the four feature categories from Iteration 1, plus one general story for navigation. They are written so someone without a security background can follow them. Story points use a 1, 2, 3, 5 scale. Our working estimate is about **2 hours per point** for our team, so a 3 point story is roughly 6 hours.

4.1 Category 1: Token Decoding and Inspection

US-1.1: Decode a pasted token

As a developer, I want to paste a JWT and see its header and payload as readable JSON so that I can check what my auth server is putting in it.

Acceptance criteria:

- Pasting a token fills the header and payload panes right away, no button needed.
- Leading and trailing spaces and a "Bearer " prefix are ignored.
- JSON is indented two spaces and keeps the original key order.
- Nothing is sent over the network.

Priority: High **Story points:** 3 **Estimate:** 6 h **Status:** Done

US-1.2: See algorithm and copy parts

As a developer, I want to see which algorithm a token uses and copy its parts so that I can paste them into a bug report or test.

Acceptance criteria:

- The algorithm from the header and the signature length are shown under the header pane.
- "Copy header" and "Copy payload" put the formatted JSON on the clipboard.

Priority: Medium **Story points:** 1 **Estimate:** 2 h **Status:** Done

US-1.3: Understand expiry at a glance

As a QA engineer, I want exp, iat, and nbf shown as real dates with a clear expired or valid label so that I can tell why a login is failing without converting timestamps.

Acceptance criteria:

- Each time claim present is listed with a label and local date and time.
- A green banner shows "Valid, expires in ..." when exp is in the future.
- A red banner shows "Expired ... ago" when exp has passed, including exactly at exp.
- An amber banner shows when there is no exp claim, and "Not valid yet" shows when nbf is in the future.

Priority: High **Story points:** 3 **Estimate:** 6 h **Status:** Done

US-1.4: Get helpful errors for bad tokens

As a developer, I want a clear message when what I pasted is not a valid token so that I know whether I copied it wrong.

Acceptance criteria:

- A token without exactly three parts says how many parts it found.
- Bad Base64URL or bad JSON says whether the header or the payload is the problem.

- The app never crashes or shows a stack trace for any input.

Priority: High **Story points:** 2 **Estimate:** 4 h **Status:** Done

4.2 Category 2: Token Encoding and Construction

US-2.1: Write claims with validation

As a developer, I want to type a header and payload in JSON and be told if I made a mistake so that I do not generate a broken token.

Acceptance criteria:

- The Encode screen starts with a sample payload whose iat is now and exp is one hour later.
- Invalid JSON blocks generation and shows the error position.
- The header can be left empty; alg and typ are filled in for me.

Priority: High **Story points:** 3 **Estimate:** 6 h **Status:** Done

US-2.2: Choose an algorithm and sign

As a developer, I want to pick HS256, HS384, HS512, RS256, RS384, or RS512 and sign with my own secret or private key so that I can make test tokens my API will accept.

Acceptance criteria:

- All six algorithms are in a drop down, HS256 selected by default.
- The chosen algorithm overrides any alg typed in the header.
- HMAC signing reproduces the well known jwt.io example token byte for byte.
- RSA signing accepts PKCS#8 PEM keys and gives a clear message for anything else.

Priority: High **Story points:** 5 **Estimate:** 10 h **Status:** Done

US-2.3: Use the generated token

As a developer, I want to copy the new token or open it in the decoder so that I can use it immediately or double check it.

Acceptance criteria:

- "Copy token" puts the token on the clipboard.
- "Open in decoder" switches screens with the token loaded.
- The token is added to recent history.

Priority: Medium **Story points:** 1 **Estimate:** 2 h **Status:** Done

4.3 Category 3: Signature Verification

US-3.1: Verify an HMAC token

As a developer, I want to check a token against our shared secret so that I can confirm it really came from our server.

Acceptance criteria:

- The correct secret shows a green "Signature verified (HS256)" banner.
- A wrong secret or any edit to the payload shows a red "does not match" banner.
- "Verify signature" on the Decode screen carries the token over.

Priority: High **Story points:** 3 **Estimate:** 6 h **Status:** Done

US-3.2: Verify an RSA token

As a QA engineer, I want to check an RS256 token with the identity provider's public key so that I can test production style tokens.

Acceptance criteria:

- A matching PEM public key verifies RS256, RS384, and RS512 tokens.
- A public key from a different key pair fails verification.
- A badly formatted key gives a message explaining the expected format.

Priority: Medium **Story points:** 5 **Estimate:** 10 h **Status:** Done

US-3.3: Trust the verification result

As a developer, I want the tool to refuse unsafe or unsupported tokens so that I never get a false "verified".

Acceptance criteria:

- Tokens with alg "none" are never verified.
- Unsupported algorithms such as ES256 are named in the message.
- An empty key asks for a secret or a public key depending on the algorithm.

Priority: High **Story points:** 2 **Estimate:** 4 h **Status:** Done

4.4 Category 4: Local Workspace and History

US-4.1: Save a token with a label

As a developer, I want to save a token with a name so that I do not have to dig it up again during a long debugging session.

Acceptance criteria:

- "Save to workspace" asks for a label and suggests the sub claim.
- A blank label becomes "Untitled token".
- Saved tokens appear in the Workspace table.

Priority: Medium **Story points:** 2 **Estimate:** 4 h **Status:** Done (mock data)

US-4.2: Find and reopen a saved token

As a QA engineer, I want to search my saved tokens and open one in the decoder so that I can quickly compare tokens for different test users.

Acceptance criteria:

- The table shows label, algorithm, status, and save time, newest first.
- Typing in the search box filters by label, ignoring case.
- Double click or "Open in decoder" loads the token.

Priority: Medium **Story points:** 2 **Estimate:** 4 h **Status:** Done (mock data)

US-4.3: Rename or delete saved tokens

As a developer, I want to rename or delete saved tokens so that my workspace stays tidy.

Acceptance criteria:

- Rename opens a dialog with the current label filled in and rejects a blank label.
- Delete removes the token and the table refreshes.

- Both ask me to select a row first if none is selected.

Priority: Low **Story points:** 1 **Estimate:** 2 h **Status:** Done (mock data)

US-4.4: See recent tokens

As a developer, I want to see the tokens I worked with recently so that I can get back to one without saving it first.

Acceptance criteria:

- Home lists up to 20 recent tokens, newest first, with no duplicates.
- Double clicking a token opens it in the decoder.

Priority: Low **Story points:** 2 **Estimate:** 4 h **Status:** Done

US-4.5: Keep my workspace after closing the app

As a developer, I want my saved tokens to still be there tomorrow so that the workspace is actually useful.

Acceptance criteria:

- Saved tokens are written to a JSON file in the user's home folder.
- The file is loaded on startup; a missing or corrupt file starts an empty workspace with a warning.
- Secrets and keys are never written to the file.

Priority: Medium **Story points:** 3 **Estimate:** 6 h **Status:** Iteration 3

4.5 Category 5: General Application

US-5.1: Move between tools in one window

As a developer, I want all the tools in one window with a side menu so that I can go from decoding to verifying without losing what I typed.

Acceptance criteria:

- A side menu reaches Home, Decode, Encode, Verify, and Workspace.
- The current screen is highlighted in the menu.
- Text typed on a screen is still there after switching away and back.

Priority: High **Story points:** 2 **Estimate:** 4 h **Status:** Done

Totals: 16 stories, 40 story points. 15 stories (37 points) are done in the prototype; US-4.5 (3 points) moves to Iteration 3.

5. Initial Prototype Overview

The prototype is a working JavaFX desktop app, not just a mockup. It covers every High priority story and most Medium ones. The spec asked for basic navigation, core features from the main stories, and a preliminary data layer that can use mock data. Here is how we met each point.

Spec item	What the prototype does
Simple UI with basic interactions	Five JavaFX screens with buttons, text areas, a drop down, a table, and dialogs. Decoding updates live as you type.
Core functionality from primary stories	Decode, inspect expiry, encode with six algorithms, and verify HMAC and RSA signatures all work for real.
Navigation between screens	A side menu switches screens. Buttons also hand a token from one screen to another (Decode to Verify, Encode to Decode, Workspace to Decode).
Preliminary database or mock data	A `TokenRepository` interface with an `InMemoryTokenRepository` preloaded with three sample tokens. A file backed version replaces it next iteration.

5.1 Running the Prototype

Requirements: JDK 17 or newer and Maven. From the repository folder:

- `mvn javafx:run` starts the app.
- `mvn test` runs the 47 JUnit 5 tests and writes a coverage report to `target/site/jacoco`.

The three sample tokens in the workspace are signed with the secret `jwtoffline-demo-secret`, so the Verify screen can be tried right away. One of them is deliberately expired to show the red banner.

5.2 Screens

The figures below are wireframes of each prototype screen. They follow the same layout, labels, and sample data as the JavaFX views in `src/main/java/com/jwtoffline/ui`.

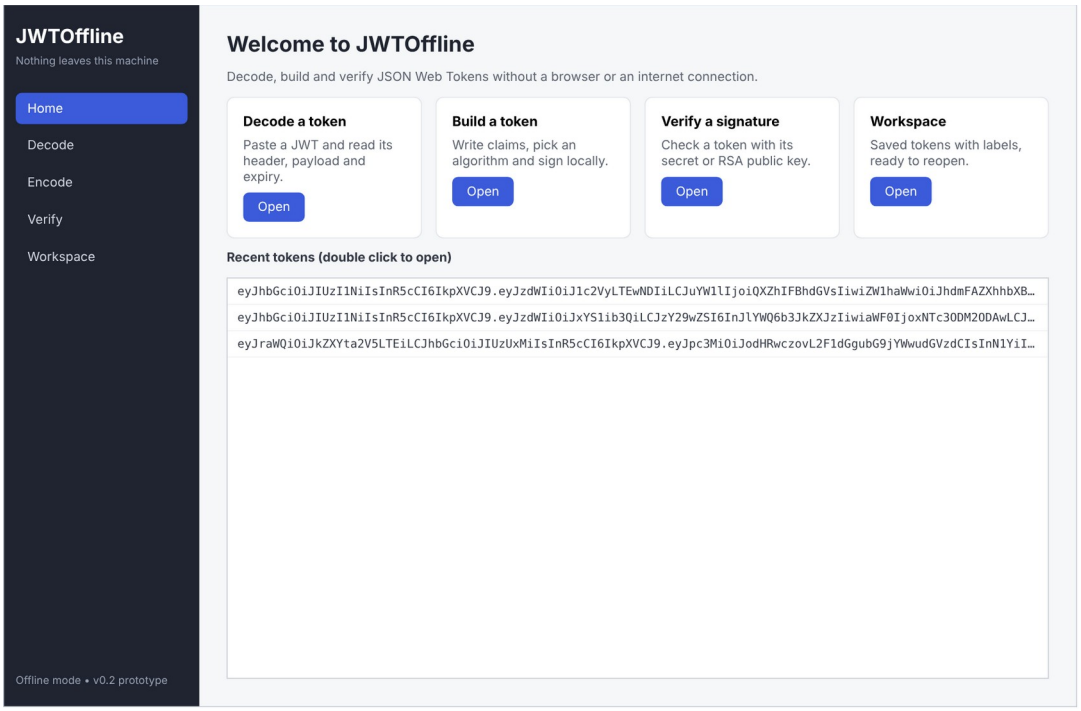


Figure 2. Home: shortcuts to each tool and the recent token history (US-4.4, US-5.1)

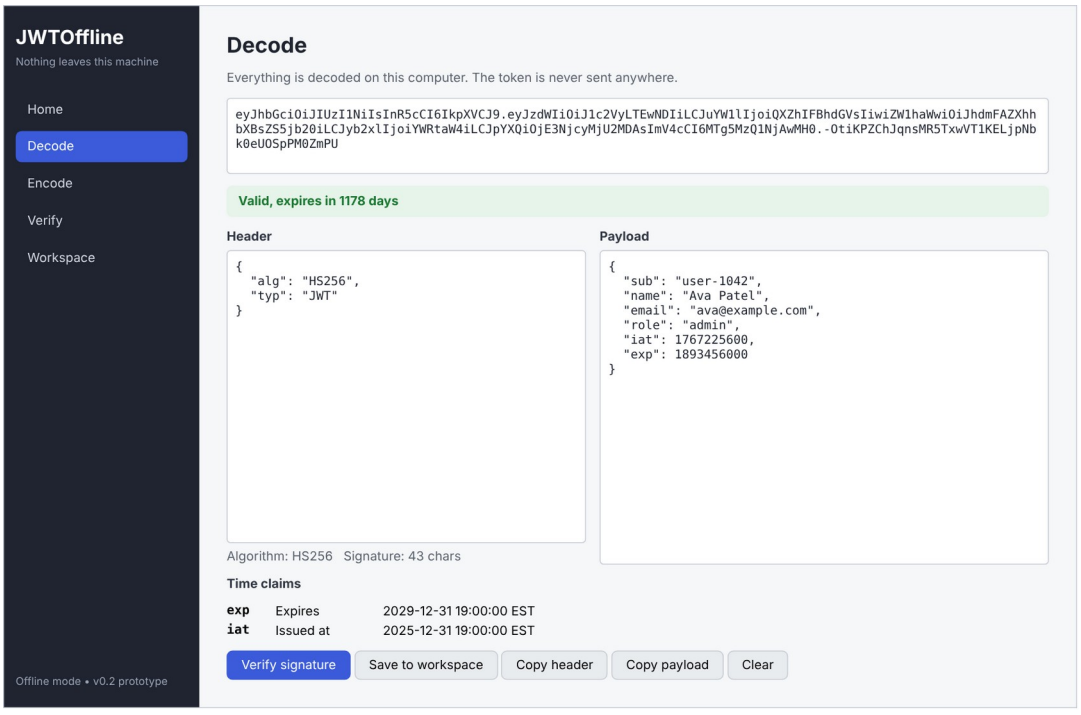


Figure 3. Decode: formatted header and payload, expiry banner, and time claims (US-1.1 to US-1.4)

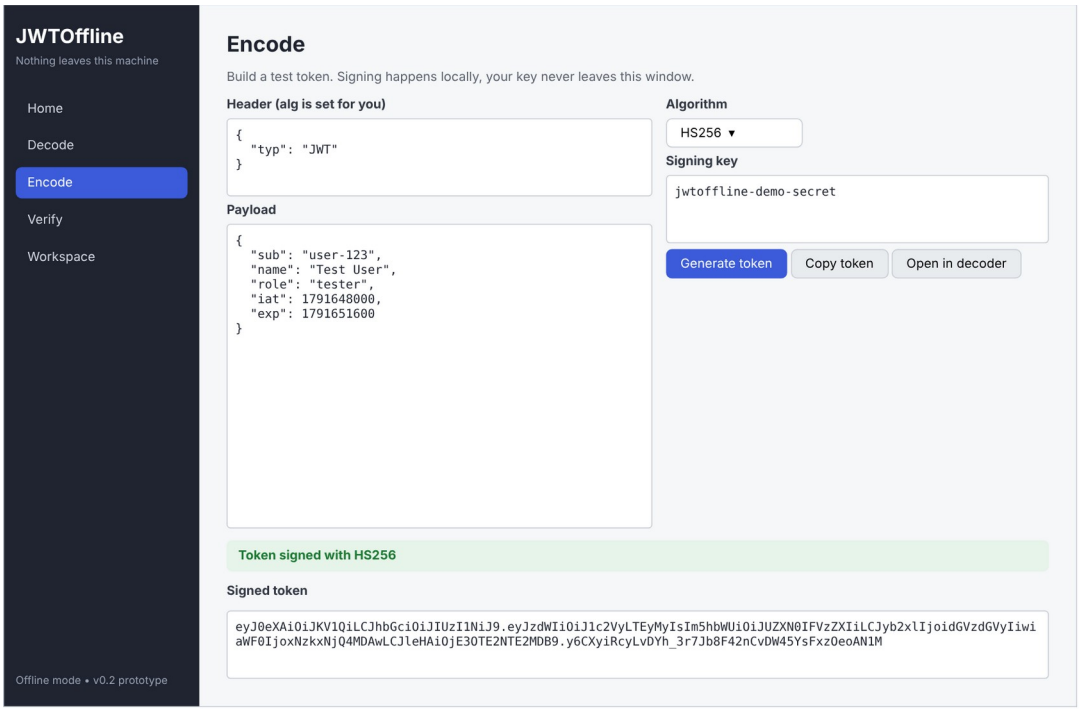


Figure 4. Encode: header and payload editors, algorithm and key, signed output (US-2.1 to US-2.3)

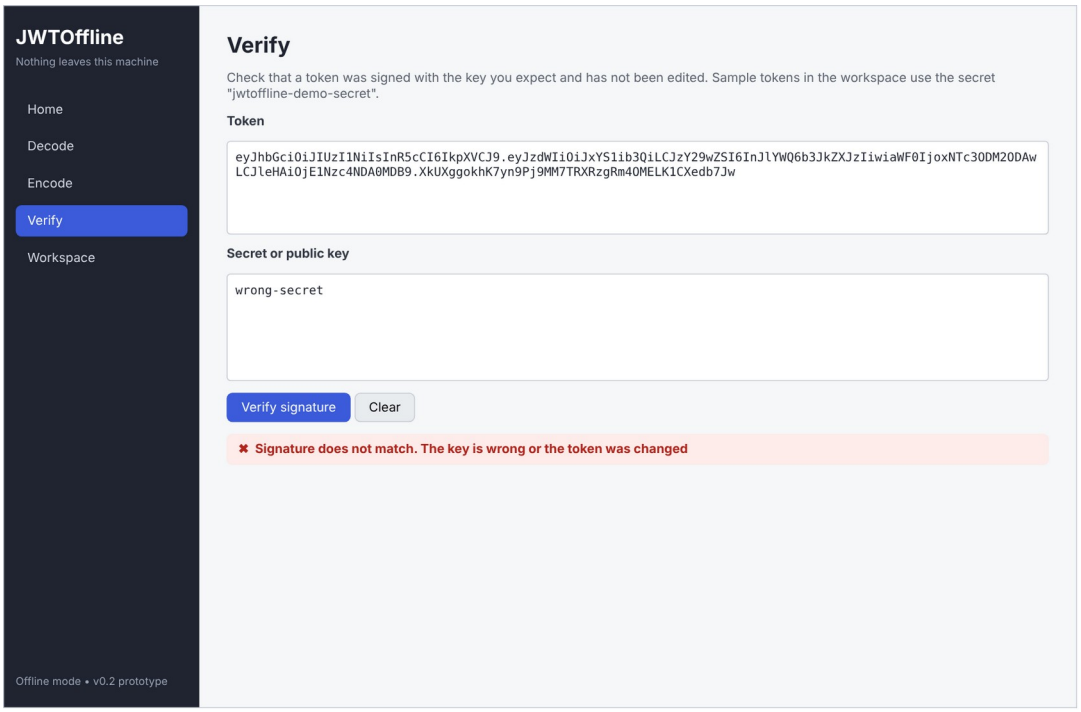


Figure 5. Verify: a wrong secret produces a clear failure message (US-3.1 to US-3.3)

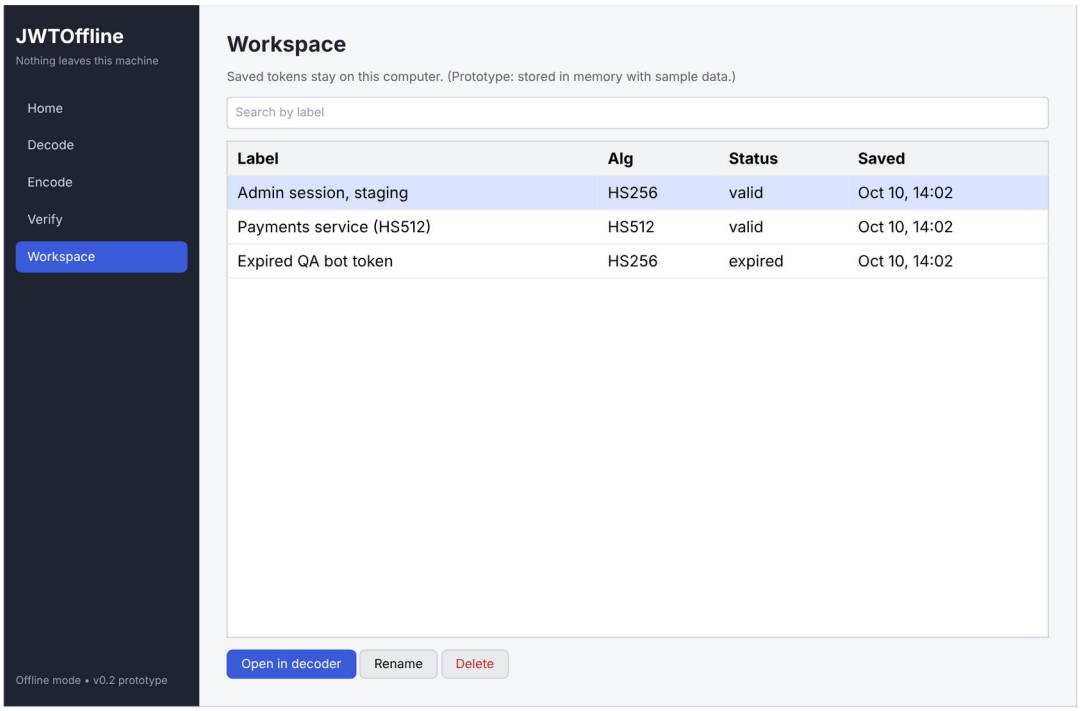


Figure 6. Workspace: saved tokens on mock data with search, open, rename, and delete (US-4.1 to US-4.3)

5.3 Testing So Far

We tested the logic layer first because that is where mistakes would actually hurt, like reporting a forged token as valid. All 47 tests pass.

Test class	Tests	What it proves
JwtDecoderTest	8	Decodes the well known jwt.io example exactly, strips "Bearer ", rejects bad part counts, bad Base64URL, and non JSON payloads with the right message.
ClaimInspectorTest	8	Valid, expired (including exactly at exp), not yet valid, and no expiry cases, using a fixed clock so results never depend on today's date.
JsonTest	4	Parses nested objects, arrays, escapes, and unicode; keeps key order; rejects broken JSON.
JwtEncoderTest	7	Reproduces the jwt.io reference token byte for byte, overrides header alg, signs with RSA, rejects bad JSON and missing keys.
SignatureVerifierTest	10	All three HMAC and all three RSA algorithms round trip, tampered payloads and wrong keys fail, alg "none" and ES256 are refused.
InMemoryTokenRepositoryTest	6	Save, search, rename, delete, blank labels, and that all sample tokens verify with the demo secret.
TokenHistoryTest	4	Newest first, no duplicates, limit enforced, blank input ignored.

A GitHub Actions workflow (`.github/workflows/build.yml`) now runs `mvn verify` on every push and pull request and uploads the JaCoCo report, so a broken build shows up before it is merged.

5.4 Developer Tasks

We split the stories into tasks small enough for one person to finish in a sitting. Hours are estimates made at sprint planning. Owners are a starting split and we track the real work on the GitHub Projects board.

Task	Description	Stories	Owner	Est. (h)	Status
T1	Package layout, App window, side navigation, AppContext	US-5.1	Isa	3	Done
T2	Base64Url helper and dependency free JSON parser and printer	US-1.1, US-2.1	Isa	4	Done
T3	JwtDecoder with user friendly errors	US-1.1, US-1.4	Steven	3	Done
T4	ClaimInspector: readable times and status	US-1.3	Steven	3	Done
T5	JwtEncoder with HMAC signing	US-2.1, US-2.2	Osmond	3	Done
T6	KeyParser for PEM keys and RSA signing	US-2.2, US-3.2	Osmond	4	Done
T7	SignatureVerifier with constant time compare and alg checks	US-3.1 to US-3.3	Isa	3	Done
T8	TokenRepository interface, in memory store, sample data	US-4.1 to US-4.3	Steven	3	Done
T9	TokenHistory	US-4.4	Steven	1	Done
T10	Decode screen	US-1.1 to US-1.4	Isa	4	Done
T11	Encode screen	US-2.1 to US-2.3	Osmond	3	Done
T12	Verify screen	US-3.1 to US-3.3	Osmond	2	Done
T13	Workspace screen with search, rename, delete	US-4.1 to US-4.3	Steven	4	Done
T14	Home screen and history list	US-4.4	Isa	2	Done
T15	JUnit tests for core and workspace	All	All	6	Done
T16	CI workflow and coverage report	NFR-7	Isa	1	Done
T17	Deliverable 1.2 document and diagrams	n/a	All	6	Done
T18	FileTokenRepository and load on startup	US-4.5	Steven	6	Iteration 3

6. Documentation Updates

6.1 System Architecture

JWTOffline is split into three packages that only depend downward. This keeps the security sensitive logic away from the UI so it can be tested on its own, and means a screen can be redesigned without touching how tokens are signed.

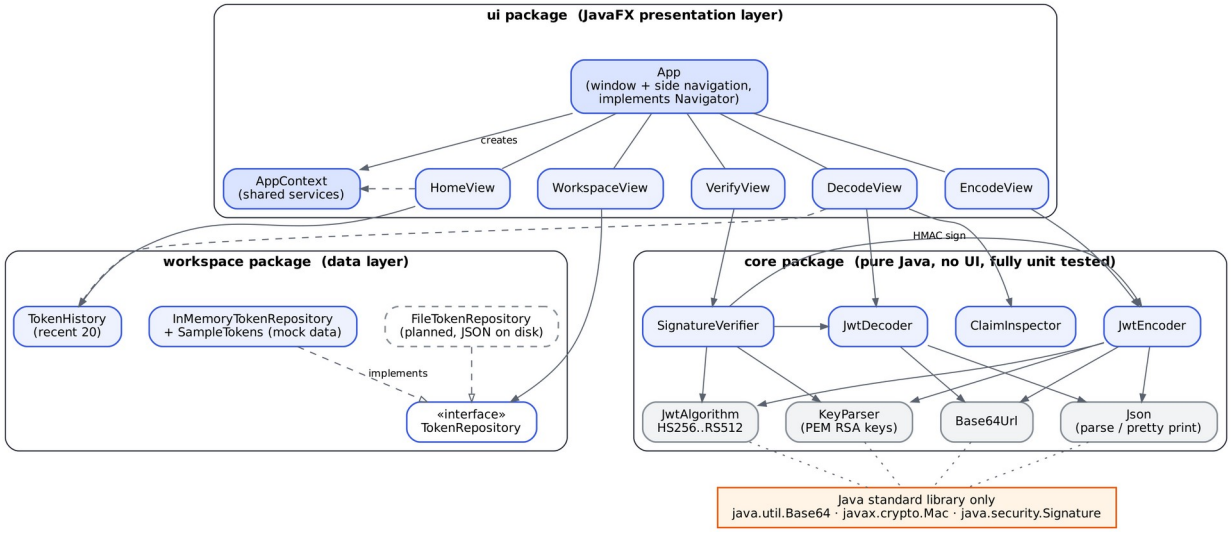


Figure 7. Package and class structure of the ITR1.2 prototype

How the components interact

- **ui package.** App builds the window and side menu and implements a small `Navigator` interface so any screen can switch to another and pass a token along. `AppContext` creates the shared services once and hands them to every screen, so screens never build their own.
- **core package.** Plain Java with no JavaFX imports. `JwtDecoder` splits and decodes, `ClaimInspector` handles dates and expiry, `JwtEncoder` signs, and `SignatureVerifier` checks signatures by reusing the decoder and the HMAC signing code. `Json`, `Base64Url`, `KeyParser`, and `JwtAlgorithm` are small helpers. Everything in this package is covered by unit tests.
- **workspace package.** Screens talk to the `TokenRepository` interface, not to a specific store. The prototype plugs in `InMemoryTokenRepository` with sample data. Next iteration a `FileTokenRepository` will be swapped in through `AppContext` with no screen changes. `TokenHistory` keeps the recent list.
- **No outside libraries at runtime.** All crypto comes from the JDK (`javax.crypto.Mac` and `java.security.Signature`). Fewer dependencies means less to audit, which matters for a tool people will trust with real tokens.

A typical request flows like this: the user pastes a token, `DecodeView` calls `JwtDecoder.decode()`, gets back an immutable `DecodedJwt`, asks `ClaimInspector` for the status, updates its panes, and adds the token to `TokenHistory`.

6.2 Planning Document and Log

Our Iteration 1 plan put requirements in Iteration 2 and all coding in Iteration 3. When we read the Deliverable 1.2 spec, it asked for a prototype now, so we pulled development forward. Both versions are below.

Original plan (from Deliverable 1.1)

Iteration	Planned work
Iteration 1 (Weeks 1 to 2)	Vision document, feature categories, GitHub repository with main and development branches.
Iteration 2 (Weeks 3 to 5)	User stories with acceptance criteria, functional and non-functional requirements, use case diagrams, prioritized backlog.
Iteration 3 (Weeks 6 to 9)	Core decoding logic in Java, JUnit tests for decoding, encoding, and validation, a prototype of the decode workflow only, testing plan and traceability matrix.

Modified plan (current)

Iteration	Planned work
Iteration 1 (Weeks 1 to 2)	No change. Completed.
Iteration 2 (Weeks 3 to 5)	Everything originally planned, plus a working prototype of all four feature categories, 47 unit tests, a traceability matrix, and a CI pipeline. Workspace runs on mock data.
Iteration 3 (Weeks 6 to 9, due Nov 8)	Formal testing plan mapped to every FR and NFR, file based workspace, performance and usability checks, raise coverage, fix feedback from this iteration.
Iteration 2.2 (due Nov 29)	Polished prototype, updated requirements from peer feedback, full documentation, presentation.

Change log

When	Change	Reason
Week 3	Moved prototype development from Iteration 3 into Iteration 2.	The 1.2 spec requires an initial prototype in this iteration.
Week 3	Prototype scope grew from decode only to decode, encode, verify, and workspace.	Encode and verify reuse most of the decode code, so adding them cost little and lets us get feedback on all features early.
Week 3	Wrote our own small JSON parser instead of adding Jackson.	Keeps the app dependency free (NFR-6, NFR-1) and the JWT JSON we handle is simple.
Week 4	Workspace uses an in memory store with sample data; saving to disk (US-4.5) moved to Iteration 3.	The spec allows mock data, and we want the storage format reviewed before we commit to it.
Week 4	Screens built in Java code instead of FXML for now.	Faster to iterate while layouts are still changing. We may move to FXML in Iteration 3.1 when the design is documented.
Week 5	Added GitHub Actions build and JaCoCo coverage report.	Catch broken builds early and give us a number to track for NFR-7.

When	Change	Reason
Week 5	Syntax highlighting of JSON moved to the backlog.	Plain formatted JSON was readable enough in testing; time went to verification instead.

7. Iteration Plan and Backlog

7.1 Iteration Plan

When	Goals
Week 3	Finalize functional and non-functional requirements. Set up packages and navigation shell. Build the decoder and JSON parser (T1 to T4).
Week 4	Finish the prototype: encoder, verifier, workspace on mock data, all five screens, unit tests (T5 to T16). Refine user stories based on what we learned while building.
Week 5	Review use cases, stories, and prototype as a team. Write this document, tag ITR1.2, submit. Collect feedback from the TA and classmates.
Weeks 6 to 9 (Iteration 3, due Nov 8)	Testing document tied to every requirement. FileTokenRepository (US-4.5). Performance timing for NFR-2. Hallway usability test for NFR-3. Fix feedback from Iteration 2.
Weeks 10 to 12 (Iteration 2.2, due Nov 29)	UI polish, syntax highlighting, keyboard shortcuts, updated documentation, and the term presentation.
Winter term (Iterations 3.1 to 3.3)	Design documentation with class and sequence diagrams (Jan 31), completed implementation including stretch features (Mar 7), final submission and demo (Mar 28).

7.2 Updated Backlog

Items are ordered by priority. Points use the same scale as Section 4.

Rank	Item	Type	Priority	Points	Target
1	US-4.5 Save the workspace to a local file and load it on startup	Story	High	3	Iteration 3
2	Fix any defects found in Iteration 2 review and feedback	Bug	High	2	Iteration 3
3	Performance tests for NFR-2 (8 KB token in under 100 ms)	Test	High	2	Iteration 3
4	Usability check with three classmates for NFR-3	Test	Medium	2	Iteration 3
5	Raise JaCoCo coverage of core and workspace above 80%	Test	Medium	1	Iteration 3
6	Syntax highlighting for header and payload JSON	Story	Medium	3	Iteration 2.2
7	Keyboard shortcuts for every action and full keyboard navigation (NFR-8)	Story	Medium	2	Iteration 2.2
8	Accept PKCS#1 RSA keys (BEGIN RSA PRIVATE KEY)	Story	Medium	3	Iteration 2.2
9	Support ES256 and ES384 (elliptic curve) signing and verification	Story	Medium	5	Iteration 3.2
10	Import a public key from a JWKS file	Story	Low	3	Iteration 3.2
11	Edit claims in a table instead of raw JSON, with a date picker for exp	Story	Low	5	Iteration 3.2

Rank	Item	Type	Priority	Points	Target
12	Dark mode	Story	Low	2	Iteration 3.2
13	Export and import the workspace	Story	Low	2	Iteration 3.2
14	Optional password protection for the workspace file	Story	Low	5	Stretch